PROJET HEATALERT · ESPRIT 5TWIN3 · ÉQUIPE DEVSYNC

Processus DevOps et pipeline CI/CD

Chaque modification du code passe automatiquement par des contrôles de qualité, des tests et une vérification de la base de données avant d'être déployée en ligne. Ce document décrit le processus mis en place et les étapes du pipeline, du développement jusqu'à la production.

Application : **Laravel 12 · PHP 8.2 · MySQL** Outil CI/CD : **GitHub Actions**

Hébergement : **Render (conteneur Docker)** Site en ligne : **heatalert-ld4p.onrender.com**

---

1 · VUE D'ENSEMBLE

Le processus DevOps du projet

Cinq développeurs travaillent chacun sur un module (alertes météo, coupures de courant, points de fraîcheur, conseils, entraide). Ils ne poussent jamais directement sur `main` : tout passe par une branche, une Pull Request et le pipeline.

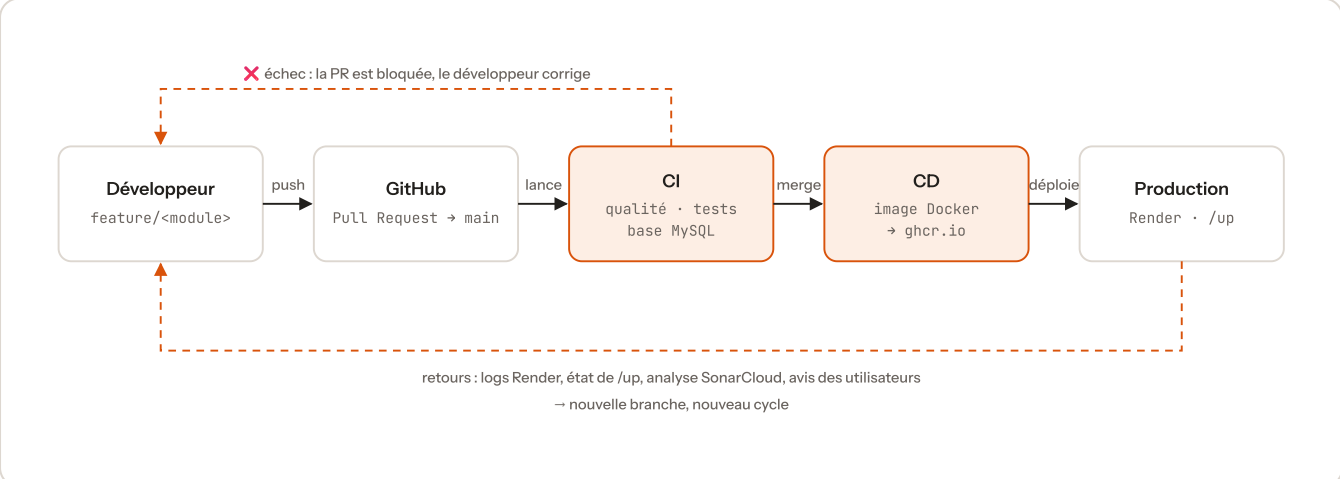


Figure 1. Une modification part d'une branche de module, traverse la CI, et n'atteint la production qu'après un merge validé. Deux boucles de retour (pointillés) renvoient vers le développeur : l'échec immédiat de la CI et les retours de production.

2 · DÉVELOPPEMENT

Le parcours d'une modification

1. **Planifier.** Chaque membre est responsable d'un module et travaille sur sa propre branche `feature/<module>` créée depuis `main`.
2. **Coder et tester en local.** `npm run dev` lance l'application, `php artisan test` lance les tests sans toucher à la base MySQL locale.
3. **Pousser et ouvrir une Pull Request** vers `main`. Le pipeline démarre automatiquement (partie CI).
4. **Lire le résultat.** Une coche verte autorise le merge ; une croix rouge indique l'étape en échec dans l'onglet *Actions*. Le développeur corrige et pousse à nouveau : le pipeline repart.
5. **Fusionner dans `main`.** Le pipeline repasse la CI sur le code fusionné, puis enchaîne la partie CD.
6. **Déploiement automatique.** L'image Docker est construite, testée, publiée, puis Render la met en ligne. Le pipeline vérifie que le site répond avant de se terminer.

L'architecture du projet limite les conflits : chaque module ajoute ses propres fichiers de routes, de menus et de tableau de bord, chargés automatiquement, au lieu de modifier des fichiers communs.

3 · PIPELINE

Les cinq étapes du pipeline CI/CD

Le pipeline est décrit dans `.github/workflows/ci-cd.yml`. Chaque étape ne démarre que si celles dont elle dépend ont réussi ; les étapes 2 et 3 tournent en parallèle pour gagner du temps.

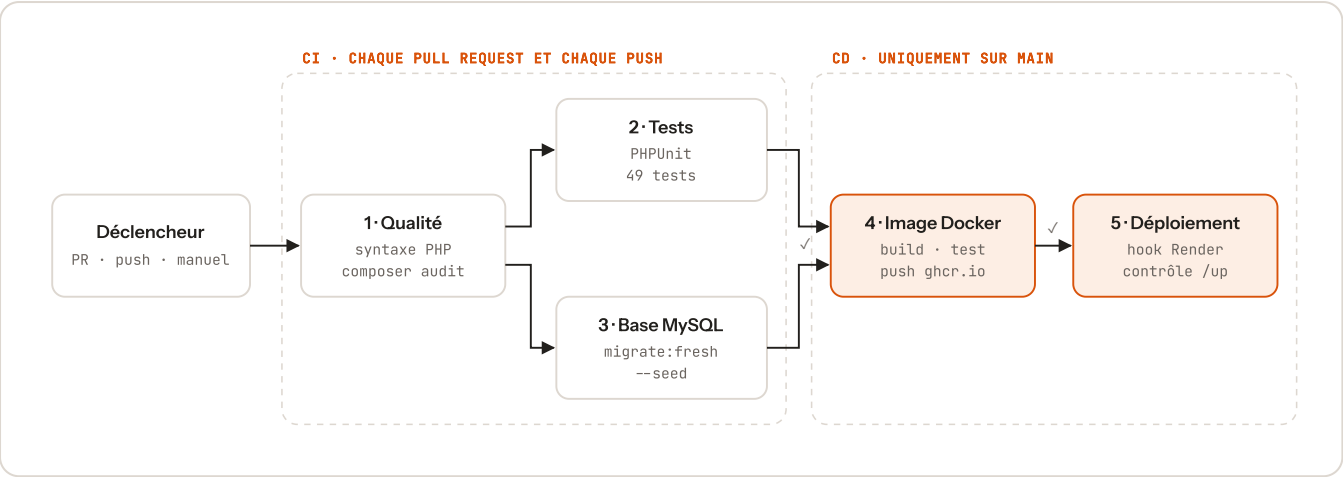


Figure 2. Sur une Pull Request, le pipeline s'arrête à la fin de la CI (étapes 1 à 3). Sur `main`, la CD (étapes 4 et 5) ne démarre que si les tests et la base MySQL ont tous deux réussi.

ÉTAPE	PHASE	CE QUI EST EXÉCUTÉ	CE QUI EST VÉRIFIÉ	LE PIPELINE S'ARRÊTE SI…
1 · Qualité & sécurité	CI	`composer validate`, `php -l` sur tous les fichiers, `composer audit`	Dépendances cohérentes, aucune erreur de syntaxe, aucune faille connue dans les bibliothèques	un fichier PHP est invalide ou une dépendance a une faille publiée
2 · Tests	CI	`php artisan test` (PHPUnit, SQLite en mémoire)	Les 36 pages des 5 modules s'affichent pour le bon profil ; CRUD, validation, signalements ; modèles d'IA	un seul test échoue
3 · Base MySQL	CI	Service MySQL 8, puis `php artisan migrate:fresh --seed`	Toutes les migrations et tous les seeders de l'équipe fonctionnent sur le vrai moteur de production	une migration ou un seeder plante (table manquante, donnée invalide…)

ÉTAPE	PHASE	CE QUI EST EXÉCUTÉ	CE QUI EST VÉRIFIÉ	LE PIPELINE S'ARRÊTE SI…
4 · Image Docker	CD	Construction du `Dockerfile`, démarrage d'un conteneur, appel de `/up` et `/coupures`, publication sur `ghcr.io`	L'application démarre réellement dans son environnement de production	le conteneur ne répond pas en 2 min 30 (l'image n'est alors pas publiée)
5 · Déploiement	CD	Appel du *deploy hook* Render, puis interrogation de `/up` sur l'URL publique	La nouvelle version est en ligne et répond	le site ne répond pas en 10 min (Render garde alors l'ancienne version)

4 · OUTILS

Outils et fichiers du pipeline

Git & GitHub
Gestion de versions, une branche par module, Pull Requests et revue avant merge.

GitHub Actions
Exécute le pipeline sur des machines Ubuntu à chaque PR et à chaque push.

Composer & PHPUnit
Installation des dépendances, audit de sécurité, tests automatisés.

SonarCloud
Analyse de la qualité et de la sécurité du code (déjà utilisée par l'équipe).

Docker
Image unique PHP 8.2 + Apache : le même environnement du test à la production.

GitHub Container Registry
Stocke chaque version de l'image, étiquetée `latest` et par numéro de commit.

Render
Héberge le conteneur, fournit le HTTPS et garde l'ancienne version si un déploiement échoue.

Open-Meteo
API météo utilisée par l'application ; les tests la simulent hors service pour vérifier que le site résiste à une panne.

FICHIER	RÔLE
`.github/workflows/ci-cd.yml`	Définition des 5 étapes, de leurs dépendances et de leurs déclencheurs
`Dockerfile`	Construction en deux temps : dépendances Composer, puis image d'exécution PHP + Apache
`docker/entrypoint.sh`	Au démarrage : mise en cache de la configuration, migrations, données de démonstration si la base est vide
`tests/` et `phpunit.xml`	Tests de fumée sur toutes les pages, tests fonctionnels du module Coupures, tests unitaires des IA

5 · SECRETS

Gestion des secrets

Aucun mot de passe ni aucune clé n'est écrit dans le code. Le fichier `.env` est exclu de Git et de l'image Docker. Les valeurs sensibles sont stockées chez GitHub et chez Render :

NOM	STOCKÉ DANS	USAGE
`GITHUB_TOKEN`	Fourni automatiquement par GitHub	Publier l'image sur ghcr.io
`RENDER_DEPLOY_HOOK_URL`	Secret GitHub Actions	Demander le déploiement à Render
`APP_KEY`	Variable d'environnement Render	Chiffrement des sessions Laravel ; le conteneur refuse de démarrer sans elle

6 · RÉSULTATS

Ce que le pipeline vérifie aujourd'hui

49	36	0	18 s
tests automatisés, 147 vérifications	pages des 5 modules contrôlées à chaque PR	faille connue dans les dépendances	pour qu'un nouveau conteneur soit prêt

Exemple concret : pendant la mise en place, une modification locale du seeder principal supprimait la création des utilisateurs. Le conteneur de test a refusé de démarrer. Dans le pipeline, l'étape 3 aurait bloqué ce changement avant tout déploiement.

7 · MISE EN PRODUCTION

Le pipeline en fonctionnement

L'application est en ligne à l'adresse https://heatalert-ld4p.onrender.com. Elle est mise à jour automatiquement à chaque fusion dans `main`. Les deux captures ci-dessous montrent la première mise en production réelle, le 31 octobre 2026.

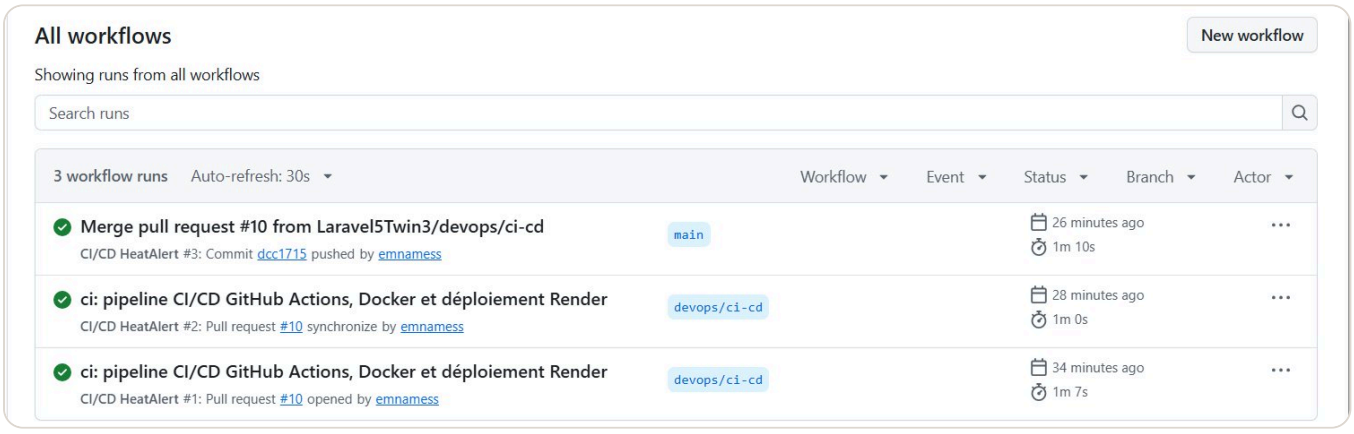


Figure 3. Onglet *Actions* de GitHub. Les exécutions #1 et #2 correspondent à la Pull Request #10 (CI seule) ; l'exécution #3 a suivi la fusion dans `main` et a enchaîné les cinq étapes jusqu'au déploiement.

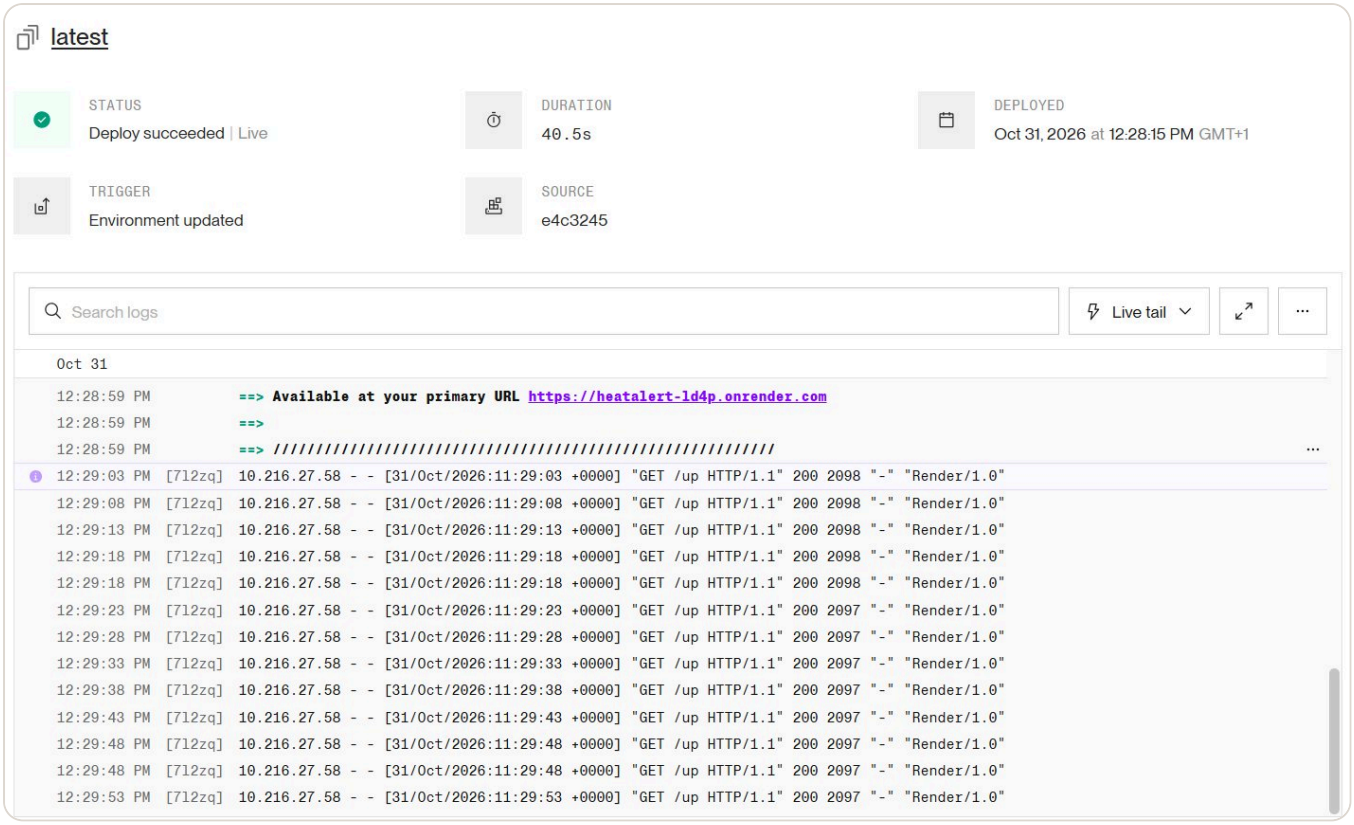


Figure 4. Tableau de bord Render après le déploiement : statut *Live*, puis le contrôle de santé `/up` qui répond `200` toutes les cinq secondes.
